

Методологии разработки ПО

От SDLC до Agile: выбираем подход для QA

Понимание методологии разработки — ключ к эффективному тестированию. На этом слайде вы изучите полный спектр подходов: от классического Waterfall до гибких Agile-методов. Каждая методология предъявляет уникальные требования к QA-процессу, критериям приёмки и временным рамкам.

Timeline Infographic



Спектр гибкости



KEY INSIGHT: Нет "универсальной" методологии. Выбор зависит от стабильности требований, размера проекта, вовлечённости заказчика и опыта команды.

SDLC: Software Development Life Cycle

Фундамент всех методологий разработки



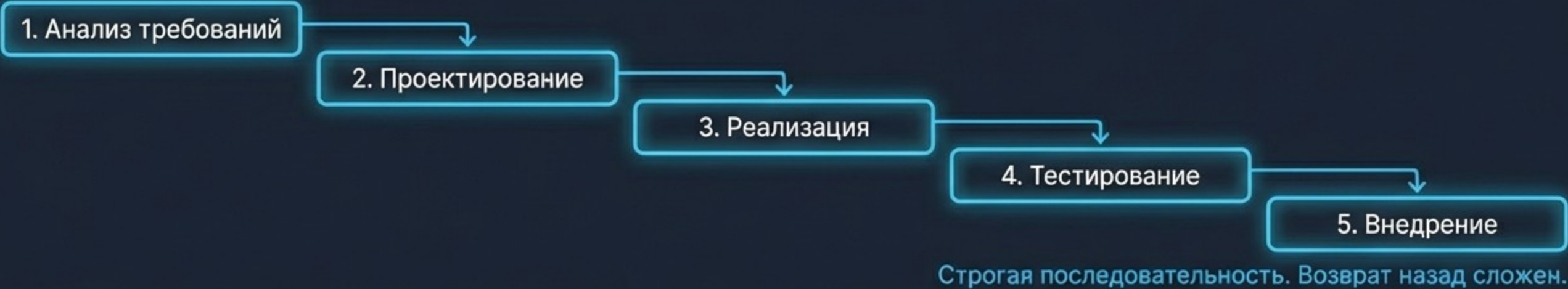
QA Activity Timeline

Фаза	QA Активность	
Концепция	Участие в обсуждениях	10%
Требования	Анализ тестируемости	30%
Проектирование	Разработка тест-плана	35%
Реализация	Подготовка тест-кейсов	40%
Тестирование	Активное тестирование	100%
Внедрение	Тестирование в prod	50%
Эксплуатация	Мониторинг дефектов	20%
Сопровождение	Регрессионное тестирование	30%

QA-INSIGHT: QA не только тестирует — QA участвует с первого дня проекта! **Раннее обнаружение дефектов = дешевле исправление.**

Waterfall Model

Классическая последовательная разработка



Параметр	Waterfall	Agile
Требования	Фиксированные	Эволюционирующие
Продукт	Виден в конце	Виден каждый спринт
Тестирование	Отдельная фаза (поздно)	На каждом спринте (рано)
Риск дефектов	Высокий (накопительный)	Средний (постоянный контроль)

REAL-WORLD: Waterfall подходит для регулируемых отраслей: Медицина, Авиация, Финансы.
Не подходит для стартапов.

V-Model

Верификация и Валидация



QA & Dev Activity Timeline



KEY DIFFERENCE: В V-Model тесты ПЛАНИРУЮТСЯ во время разработки, а выполняются после.

Spiral Model

Управление рисками в крупных проектах

Spiral Diagram

1. Планирование

2. Анализ рисков

3. Разработка и
Тестирование

4. Оценка и
Планирование
следующего витка



Каждый виток = снижение неопределённости

Risk vs Time Chart

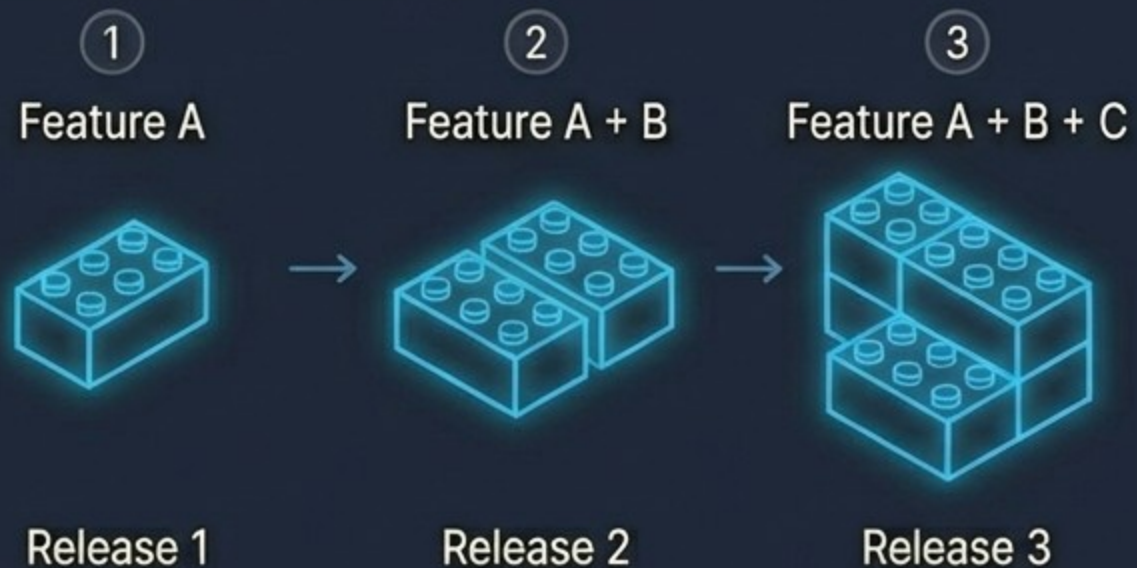


REAL-WORLD EXAMPLE: NASA, Военные системы. Используют Spiral Model для проектов с **ВЫСОКОЙ СТОИМОСТЬЮ ОШИБКИ.**

Incremental vs Iterative

Стратегии эволюции продукта

Incremental (По частям)



JetBrains Mono

QA тестирует новую фичу + регрессия старых.

Iterative (Черновики)



Iteration 1 (v0.1)

Iteration 2 (v0.5)

Iteration 3 (v1.0)

JetBrains Mono

QA тестирует улучшения существующего функционала.

Incremental QA

Большие регрессионные наборы

Функционал добавляется

Iterative QA

Фокус на исследовательском тестировании

Функционал изменяется

RAD Model

Rapid Application Development

RAD Cycle Diagram



Timeline Comparison



BEST FOR:

Внутренние инструменты,
Прототипы, MVP.

NOT FOR:

Критически важные
системы со сложной
архитектурой.

Сравнение традиционных подходов

Шпаргалка для выбора

Методология	Гибкость	Документация	Риски	Тип проекта
Waterfall	Низкая	Высокая	Высокие	Чёткие требования
V-Model	Низкая	Высокая	Средние	Высокая надёжность
Spiral	Средняя	Высокая	Управляемые	Дорогие/Сложные
RAD	Высокая	Низкая	Средние	Срочные/MVP
Agile	Очень высокая	Минимальная	Низкие	Неопределённость



Agile Manifesto

4 Ценности: Люди и продукт важнее процессов



Люди и взаимодействие
важнее процессов
и инструментов



Работающий продукт важнее
исчерпывающей
документации



Сотрудничество с заказчиком
важнее согласования
условий контракта



Готовность к изменениям
важнее следования
плану



TRANSFORMATION:

QA перестаёт быть 'вратарём' и становится 'партнёром'.

12 Принципов Agile

Практическое руководство

1  Удовлетворение клиента

2  Приветствие изменений

3  Частая поставка

4  Бизнес и Dev вместе

5  Мотивированная команда

6  Личное общение

7  Работающий продукт (мерило прогресса)

8  Устойчивый темп

9  Техническое совершенство

10  Простота (не делать лишнего)

11  Самоорганизация

12  Регулярная рефлексия (Ретро)

QA IMPACT: Гибкие планы, ежедневное общение, автотесты, постоянное улучшение процессов.

Scrum Framework

Роли, Артефакты, События

Роли

-  Product Owner
-  Scrum Master
-  Team (Dev + QA)

Артефакты

-  Product Backlog
-  Sprint Backlog
-  Increment (Working Soft)

События

-  Planning
-  Daily Standup
-  Review
-  Retrospective



QA IN SCRUM:

QA участвует в оценке задач (Poker) и тестирует внутри спринта. Нет отдельной фазы "тестирование" после спринта.

Kanban

Визуализация потока и WIP-лимиты



****FLOW:**** Ограничение незавершённой работы (WIP) ускоряет прохождение задач. Идеально для поддержки (Support).

Scrum vs Kanban

Детальное сравнение

МАТРИЦА СРАВНЕНИЯ

Характеристика	Scrum	Kanban
Итерации	Спринты	Поток
Роли	Есть	Нет
Изменения	В след. спринте	В любое время
Оценка	Story Points	Lead Time
Метрики	Velocity	Cycle Time

ДЕРЕВО РЕШЕНИЙ



STATISTICS: 70% команд используют Scrum, 20% Kanban, 10% Гибрид.

Роль и стоимость тестирования

Waterfall vs Agile

Waterfall

[Dev]

[Test]

QA "находит" баги в конце. Стресс.

Agile

[Dev/Test]

[Dev/Test]

[Dev/Test]

[Dev/Test]

[Dev/Test]

QA предотвращает баги постоянно.

Cost of Defect Chart



****CRITICAL INSIGHT:** Чем позже найден дефект, тем дороже его исправить.
Agile сдвигает тестирование влево (Shift Left).

Выводы: Выбор методологии

Матрица решений для QA



Медицина
(FDA)
-> **V-Model**



Стартап ->
Agile/Scrum



Авиация
-> **Spiral**



ERP Система
-> **RAD/Scrum**



Поддержка
-> **Kanban**



Криптография
-> **V-Model**



Мобильное
App -> **Agile**



DevOps Tools
-> **Kanban**

Советы Архитектора:

- ✓ Поймите контекст (цена ошибки).
- ✓ Не бойтесь менять процесс.

- ✓ Измеряйте метрики качества.
- ✓ Будьте партнером, а не контролером.